Final Project: Submission and Evaluation

Latest attempt: 3/31/2026, 3:21:45 PM See all attempts

Resume

About this assignment

Assignment type	Time Limit	Estimated Time	Assignment attempts	Passing Grade
Graded	Unlimited	30 minutes	Unlimited	75%

For this final project – Emotion Detector, you have completed the tasks below while assuming the role of a software engineer developing an AI-based web application.

- Task 1: Clone the project repository
- Task 2: Create an emotion detection application using the Watson NLP library
- Task 3: Format the output of the application
- Task 4: Package the application
- Task 5: Run Unit tests on your application
- Task 6: Web deployment of the application using Flask
- Task 7: Incorporate error handling
- Task 8: Run static code analysis

As part of your submission, you will be asked to submit your input. The evaluator will provide feedback on your assignment.

Instructions

To complete this assignment successfully, follow these steps:

- Complete all the tasks from the Final Project - Emotion Detector
- Ensure that all tasks are completed, with the repository URL, terminal outputs, ar ippets saved, and screenshots captured.
- Select 'Begin the Assignment' to get started.
- Read each question and submit your response by providing a URL, code snippet, terminal output, or screenshots taken for the project.
- Once you've answered all the questions, select 'Submit assignment.'
- Review the feedback provided to understand your performance and areas for improvement.

Grading Criteria

You will be graded as per the following rubric (16 points total; requires 75% or 12 points to pass):

Task 1: Submit the GitHub repository URL
Submit the public GitHub repository URL of the README.md file, which contains the project name details. **(1 point)**

Task 2: Create an emotion detection application using the Watson NLP library
Activity 1: Submit the code from the emotion_detection.py file that shows the application function. **(1 point)**
Activity 2: Submit the terminal output showing that the application was imported and tested without errors. **(1 point)**

Task 3: Format the output of the application
Activity 1: Submit the code from the emotion_detection.py file showing that the modified emotion_detector function returns the correct output format. **(1 point)**
Activity 2: Submit the terminal output showing that the output format is accurate. **(1 point)**

Task 4: Validate the EmotionDetection package
Activity 1: Submit the public GitHub repository URL of the __init__.py file which includes the code to import the application module. **(1 point)**
Activity 2: Submit the terminal output validating that EmotionDetection is a valid package. **(1 point)**

Task 5: Run unit tests on your application
Activity 1: Submit the code from the test_emotion_detection.py file showing the required unit tests. **(1 point)**
Activity 2: Submit the terminal output showing that all unit tests passed. **(1 point)**

Task 6: Web deployment of the application using Flask
Activity 1: Submit the code from the server.py file showing the web deployment of the application using Flask**. (1 point)**
Activity 2: Upload the screenshot named 6b_deployment_test.png showing the application deployment. **(1 point)**

Task 7: Incorporate error handling
Activity 1: Submit the code from the emotion_detection.py file showing the updated emotion_detector function for status code 400. **(1 point)**
Activity 2: Submit the code from the server.py file showing the handling of blank input errors. **(1 point)**
Activity 3: Upload the screenshot named 7c_error_handling_interface.png validating error handling functionality. **(1 point)**

Task 8: Run static code analysis
Activity 1: Submit the code from the server.py file demonstrating the execution of static code analysis. **(1 point)**
Activity 2: Submit the terminal output showing a perfect score for static code analysis. **(1 point)**

Resume